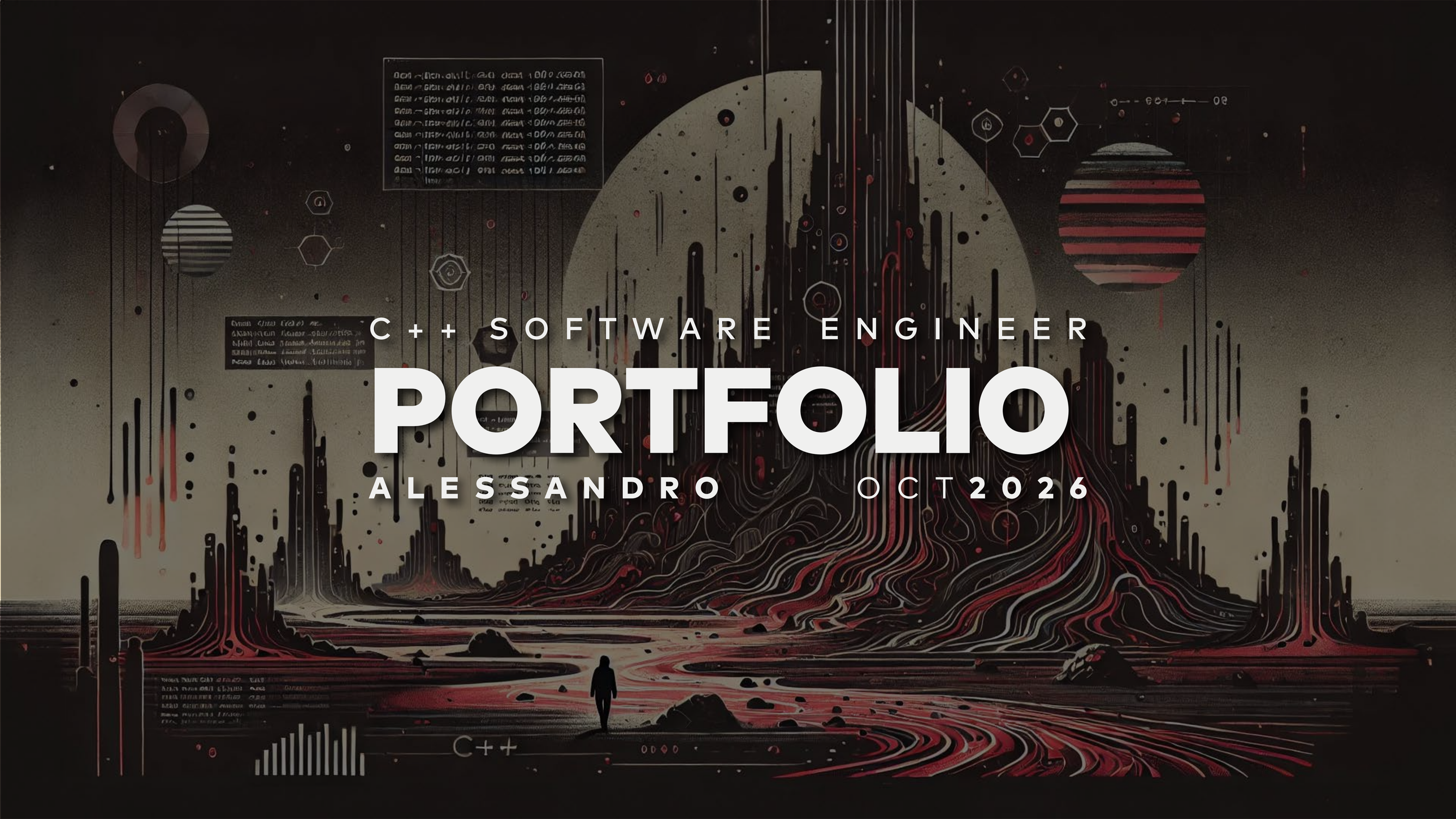
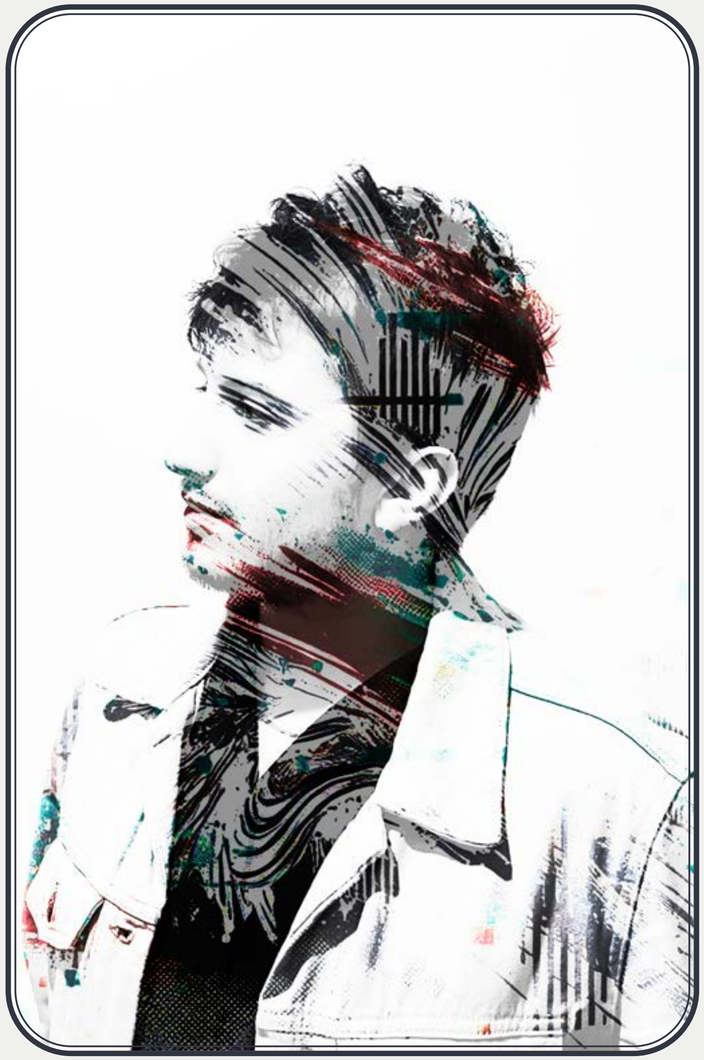

C++ SOFTWARE ENGINEER
PORTFOLIO
ALESSANDRO
OCT2026
C++

Ciao!

My name is Alessandro Manzini and I'm a ***C++ software engineer*** with a strong focus on modern practices and experimental thinking.

While ***engine development*** and ***graphics programming*** are my biggest passions, I genuinely enjoy and thrive in every software development field.

I'm always excited at the thought of working with professionals who are passionate about their craft — to ***learn***, to sometimes ***teach***, and most importantly to ***build*** something meaningful together.



EXPERIENCE

Feb 2026 - Jun 2026

C++ Engine/Graphics Intern at *Triumph Studios*

Jan 2020 - Nov 2024

Full-stack C# Dev at ***Mpt Srl***

COMPETENCES

EDUCATION

2023 - 2026: Howest — Digital Arts & Entertainment

Game Development Bachelor Degree

2020 - 2020: University of Parma

Space Invaders in Python course

2014 - 2019: Inst. of Higher Education "Blaise Pascal"

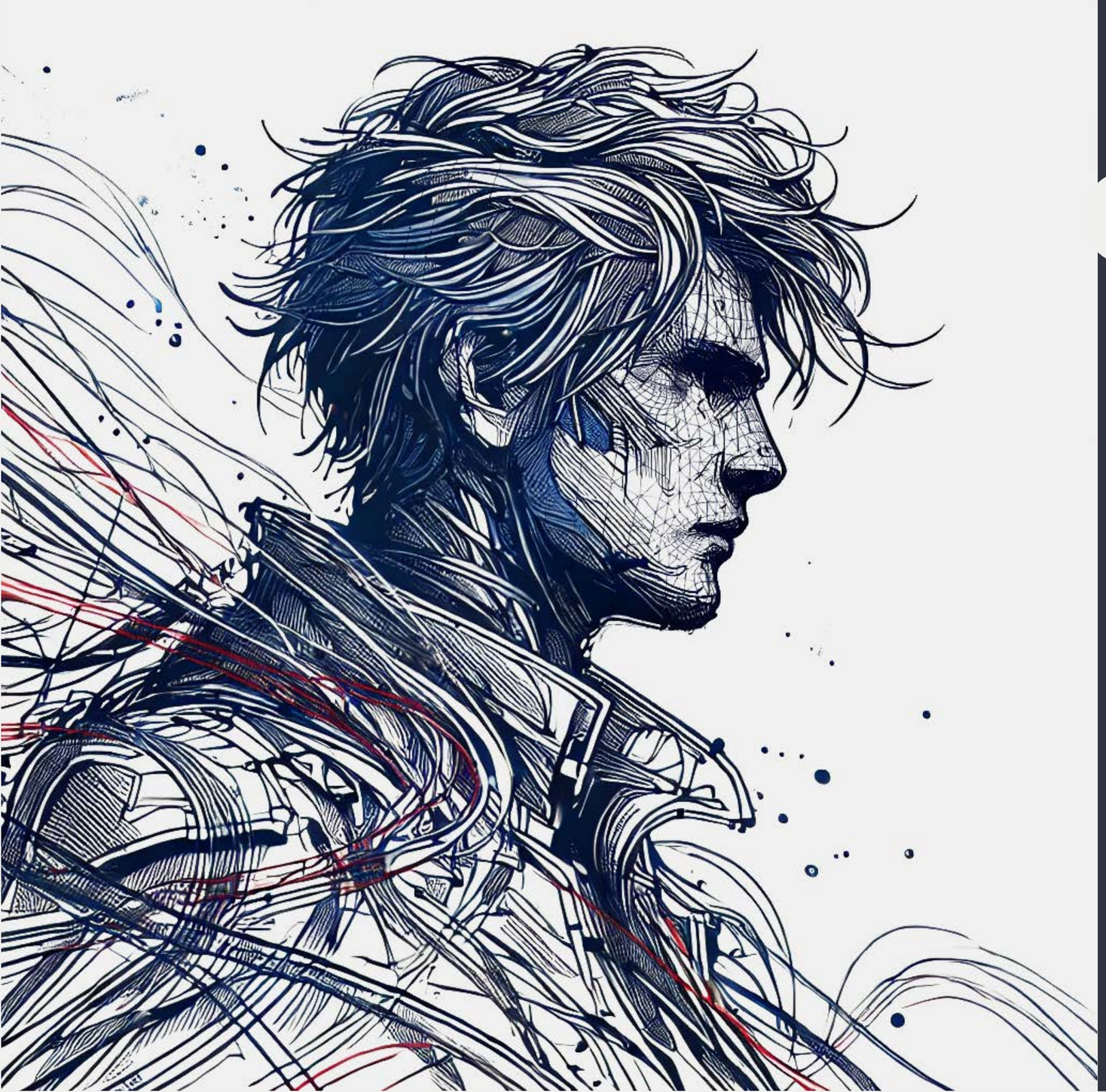
Technical Diploma on IT and Telecommunications

/in/alessandro-manzini-developer/

github.com/alessandromanzini

a.a.manzini00@gmail.com

+39 331 608 11 94



CONTENTS

01 WORK EXPERIENCE

TRIUMPH STUDIOS FEB – JUN 2026 C++ Engine & Graphics Intern ↗

02 PERSONAL PROJECTS

ASH LIBRARY An OS native diagnostic library ↗

RHASTER ENGINE A cross-platform 2D game engine ↗

VULKAN RASTERIZER A cross-platform Vulkan renderer ↗

CUPHEAD REMAKE My PROG2 exam project ↗

GEOMETRIC ALGEBRA PHYSICS A physics engine based on PPGA ↗

03 ABOUT ME

CHARACTER & VALUES Who I am and how I work ↗

RECOMMENDATIONS Professors and employers who vouch for me ↗

ENGINE DEV INTERNSHIP

C++ ENGINE & GRAPHICS INTERN · FEB – JUN 2026

Five months on the engine team at ***Triumph Studios***, the Paradox studio behind ***Age of Wonders***, working on an unannounced title. My main task was rebuilding the engine's ***overlay renderer***, the designated system for debug views, world-space markers and gameplay UI, from the C++ API down to ***DirectX 12 mesh shaders***.

Geometry is now built ***entirely on the GPU***, effects and blend modes run as ***compute passes***, and repeated draws are cached in GPU memory. I also wrote a CAD-style ***snap system*** for the spline construction tools.

40×

faster than the old overlay renderer in a stress test (1 → 40 FPS)

2.9×

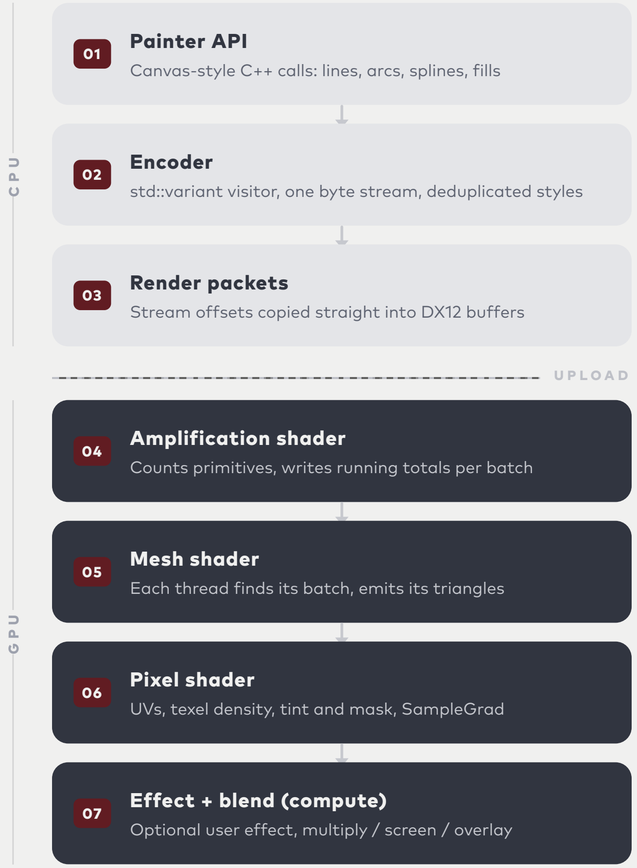
less CPU time after one profiling sprint (364 → 125 ms)

> *"We gave him a difficult task, outside his comfort zone, and he managed it nicely."*
>
> SUPERVISOR · MIDTERM FEEDBACK

> *"I think he has a strong understanding of architecture and usability."*
>
> COLLEAGUE · SNAP SYSTEM DELIVERABLES

THE OVERLAY RENDERER

From one C++ call to pixels, in one frame



TRIUMPH STUDIOS

batches.hlsl

```
struct BatchPayload
{
    uint Total[MAX_BATCHES]; // running totals: sorted
    uint BatchCount;
};

// Every mesh-shader thread finds its own batch
// by binary-searching the running totals.
uint FindBatch( in BatchPayload p, uint primitive )
{
    uint lo = 0, hi = p.BatchCount;
    while ( lo < hi )
    {
        uint mid = ( lo + hi ) / 2;
        if ( p.Total[mid] <= primitive ) lo = mid + 1;
        else hi = mid;
    }
    return lo;
}

[outputtopology( "triangle" )]
[numthreads( 64, 1, 1 )]
void MainMS( uint thread : SV_GroupIndex,
             uint3 group : SV_GroupID,
             in payload BatchPayload p,
             out vertices Vertex verts[256],
             out indices uint3 tris[128] )
{
    // One quad per thread; the last group may be partial.
    uint base = group.x * 64;
    uint quads = min( 64, p.Total[p.BatchCount - 1] - base );
    SetMeshOutputCounts( quads * 4, quads * 2 );
    if ( thread >= quads ) return;

    uint primitive = base + thread;
    uint batch = FindBatch( p, primitive );
    uint first = batch > 0 ? p.Total[batch - 1] : 0;
    EmitQuad( batch, primitive - first, thread, verts, tris );
}
```

*Simplified illustration of the batch lookup, not NDA studio code.

TRIUMPH STUDIOS

WHAT'S UNDER THE HOOD

CPU TO GPU

One byte stream, sized before it's written

Draw calls, geometry and deduplicated styles serialize into ***one big staging stream***; render packets keep offsets from it and copy data straight into ***DX12 buffers***. The stream capacity is constantly adjusting: I predict the ***exact*** requirements for geometry and styles, ***worst-case*** for draw calls, then average on ***last frames usage*** to avoid over-reservation.



AMPLIFICATION TO MESH

Work distribution without the CPU

The amplification shader writes a ***running*** (async) primitive total per batch and then instantiates the needed amount of ***mesh-shader*** threads. Each thread binary-searches the running sum to find its batch and primitive. From there it generates geometry and UVs completely in ***parallel***, ***even*** for dynamically defined shapes like ***splines***.

POST-PROCESSING

A pluggable effects stage

Overlays render into a ***transient image***, an optional compute pass runs a ***user effect*** (material shader), and a blend ***compute pass*** (multiply, screen, burn or overlay) combines it with the color attachment. If neither an effect nor a blend is requested, a ***fast path*** skips everything.

RENDERING PIPELINE

Artefacts and bug fixes

A ***MIP seam*** at UV wraps, fixed with ***SampleGrad*** on the unwrapped UVs instead of forcing MIP 0. Overlays jittering under ***DLSS***, fixed by removing the temporal jitter from the clip position: a bug RenderDoc hides, since it disables upscaling.

ASH, MY DIAGNOSTIC LIB

This project started from the necessity of having reliable tools I could reuse over my projects.

The first thing I did was creating an ***OS native modal view*** that would help display assertions or other error messages. This has taught me much about iOS Objective-C and is by itself a display of project management, but what came after is the highlight of this project.

I would argue that the most useful tool for diagnostics is a logger. On the surface, it's something quite easy to implement, but I decided to over-engineer the heck out of it ***for fun***. I wanted to make it ***flexible*** enough to suit all kinds of uses, but also ***faster*** than any application would ever need.

The ***git history*** on this project (specifically on the logger branch) is a long list of ideas I experimented with and I improved upon:

I wanted zero heap allocations, so I created a ***chunking iterator*** for the ***std::format_to*** function; I wanted an efficient stream write, so I designed a ***char buffer*** that sinks on the stream only with big chunks of text; I wanted a ***lockless worker thread***, so I made a ***deferred mode***; then scrapped half of the codebase because it was still 100x slower than ***Quill***, to then re-design the system with a format argument serializer so I wouldn't have to process the string on the caller thread. The list goes on...

Check it out on **GITHUB**!

ASH LIBRARY

main.cpp

```cpp
import ash;

ash::Logger logger{ ash::policy::deferred_sync, {
    .identity     = "render",
    .title_format = "{TIME:HH:MM:SS.mmm} {SIG:u} {ID}",
} };

// Checked at compile time, like std::format.
logger.info( "frame {} took {:.2f} ms", frame, ms );

// Call-site modifiers chain onto any call.
logger.warning( "missing asset {}", name ).once( );
logger.debug( "tick {}", tick ).every( 100 );
logger.info( "fps {}", fps ).throttle( 1s );
logger.error( "retry {}", n ).at_most( 3 );
logger.trace( "state {}", s ).when( verbose );

// Native modal dialogs, e.g. for assertions.
auto const choice = ash::Modal{ "Assertion" }
    .with_desc( ash::Description::as_body( msg ) )
    .with_choice( ash::Choice::as_master( "Ignore", 0 ) )
    .with_choice( ash::Choice::as_cancel( "Exit", 1 ) )
    .open( );
```

OUTPUT

```
12:04:51.382 INFO render
frame 1042 took 6.94 ms
12:04:51.383 WARNING render
missing asset hero.png
```

ASH LIBRARY

SOME LOG BENCHMARKS

I've put a lot of passion in this project, and I'm extremely ***proud*** of the ***results***. I benchmarked my library against the leading ***open-source*** loggers, outperforming ***spdlog*** by two orders of magnitude and even ***quill*** in some tests.

4–14 ns

median per deferred call, from integer to string messages

2.2×

faster encoding for strings compared to quill

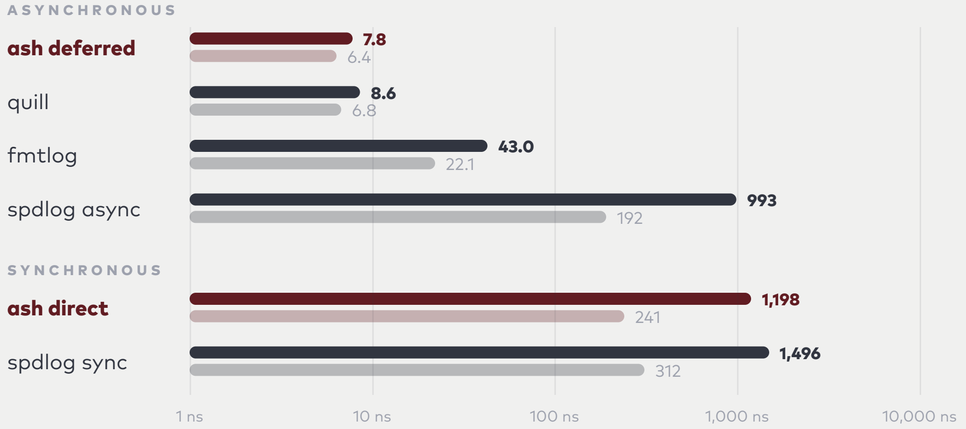
0

locks or system calls on the deferred path; a full ring drops entries

CALLER-SIDE LATENCY PER LOG CALL

Bursty · Hot

Median ns on the calling thread, geometric mean of three message types · lower is better · log scale
Bursty: the core idles between 32-call bursts · Hot: it stays busy · same prefix and /dev/null sink for all



APPLE SILICON M2 · clang 22.1.8 -O3 -DNDEBUG · spdlog 1.17.0 · quill 13.1.0 · fmtlog feb2025

Check it out on GITHUB!



MY OWN 2D GAME ENGINE

I started on this project for my Programming 4 exam, but quickly got addicted at making this a safe, modern and optimized tool.

This has been one of my favorite projects, teaching me much about performant low level coding and designing optimal interfaces for other engineers to build upon.

This is vastly an unfinished project (turns out programming an engine takes some time) but it still features some interesting ideas.

I'm planning to redesign this whole engine from scratch in my ***firefly engine*** project. It will have an improved modular architecture, 3D space, and a custom renderer attached to it.

KEY FEATURES

- **Modern C++23**
- **Cross platform**
- **ECS architecture** sparse sets
- **Scene graph**
- **Input binding** modifiers, triggers
- **Parallel audio queue**
- **Single object process** scoped services, no singletons

RHASTER ENGINE

NOTES

1. My ECS uses ***sparse sets*** under the hood to store components.

2. I didn't want components to inherit from an interface, therefore I applied exclusively ***polymorphism to the components container***. Then, using a ***map*** of type-erased sparse sets against a ***compile-time hash*** of the components' type.

This would be an issue in case of component inheritance from the user, but — traditionally — components should not inherit in the first place (or at the very least they shouldn't query the map using base-type hashes).

ENTITY COMPONENT SYSTEM

Referencing EnTT blog posts, I built an ***ECS architecture*** prioritizing ***modern C++*** principles, such as the proactive use of ***templates*** and ***compile-time paradigms***.

registry.h

```
class registry final
{
public:
    [[nodiscard]] auto entity_alloc( ) -> entity_allocator&;

    template <detail::ecs_component TComponent, typename... TArgs>
        requires std::constructible_from<TComponent, TArgs...>
    auto emplace( entity_type const entity, TArgs&&... args ) -> TComponent&;

    template <detail::ecs_component TComponent>
    auto remove( entity_type const entity ) -> void;

    template <detail::viewable_ecs_component... TComponents>
    [[nodiscard]] auto has( entity_type const entity ) const -> bool;

    template <detail::viewable_ecs_component... TComponents>
    [[nodiscard]] auto view( ) -> view<TComponents...>;

private:
    std::unordered_map<meta::hash::hash_type, unique_ref<detail::base_reg_pool_type>> pools_{};
    entity_allocator entity_alloc_{};

    template <detail::viewable_ecs_component TComponent>
    [[nodiscard]] auto ensure_pool( ) -> detail::reg_pool_type<TComponent>&;
};
```

NOTES

1. Since a view is an ***immutable*** and ***independent*** object, I can adopt a more template heavy design.

I store sparse sets references as tuples and apply concept constraints for type safety.

2. The view iterates through the ***smallest set***, using a custom iterator that stops on entities that own all components.

3. Views are about ***lazy evaluation*** during iteration. Memory efficient, but slower due to ***poor cache locality***.

In the future I plan to implement a more performant ***"group"*** structure for ***hot code paths***.

RHASTER ENGINE

VIEW ACCESS IN THE ECS

In order to access components and related entities, I created a ***view***. A ***view*** is a simple intersection of sparse sets, aiming to provide an ***iterator*** over all entities that ***share*** the selected ***components***.

view.h

```
template <detail::viewable_ecs_component... TComponents>
    requires ( sizeof...( TComponents ) > 0U )
class view final
{
public:
    explicit view( detail::reg_pool_type<TComponents>&... pools ) noexcept;

    template <meta::contains_type<TComponents...>... UComponents>
    auto has( entity_type const entity ) const -> bool;

    template <typename... UComponents>
        requires ( meta::contains_type<UComponents, TComponents...> && ... )
    auto get( entity_type const entity ) -> view_get_type<UComponents...>;

    template <meta::contains_type<TComponents...>... UComponents>
    auto unsafe_get( entity_type const entity ) -> underlying_view_get_type<UComponents...>;

    template <std::invocable<entity_type, TComponents&...> TDelegate>
    auto each( TDelegate&& delegate ) noexcept(std::is_nothrow_invocable_v<TDelegate>) -> void;

    template <std::invocable<TComponents&...> TDelegate>
    auto each( TDelegate&& delegate ) noexcept(std::is_nothrow_invocable_v<TDelegate>) -> void;

    auto each( ) noexcept -> view_range<view>;

    [[nodiscard]] auto begin( ) -> sparse_intersection_iterator<entity_type, TComponents...>;
    [[nodiscard]] auto end( ) -> sparse_intersection_iterator<entity_type, TComponents...>;

private:
    std::tuple<detail::reg_pool_type<TComponents>&...> const pools_;
    detail::base_reg_pool_type& smallest_pool_ref_;
};
```

main.cpp

```cpp
// create entities using the entity allocator
auto entity1 = registry.entity_alloc( ).create( );
auto entity2 = registry.entity_alloc( ).create( );

// add components
registry.emplace<position>( entity1, 10.0f, 20.0f );
registry.emplace<velocity>( entity1, 1.0f, 0.5f );

// basic view usage for system processing
auto view = registry.view<position, velocity>( );

// method 1: lambda-based iteration with entity ID
view.each( []( entity_type entity, position& pos, velocity& vel ) {
    pos.x += vel.dx;
    pos.y += vel.dy;
} );

// method 2: lambda-based iteration without entity ID
view.each( []( position& pos, velocity& vel ) {
    pos.x += vel.dx;
    pos.y += vel.dy;
} );

// method 3: range-based for loop with structured bindings
for ( auto [entity, pos, vel] : view.each( ) ) {
    pos.x += vel.dx;
    pos.y += vel.dy;
    // entity is the entity ID, pos and vel are component references
}

// component subset access (get only position from position+velocity view)
for ( entity_type entity : view ) {
    auto pos_result = view.get<position>( entity );
    if ( pos_result ) {
        position& pos = pos_result.value( );
        // process only position component
    }
}
```

RHASTER ENGINE

ABOUT COMPONENTS

At last, there's a design consideration to be made about components.

Traditionally, ECS known practices want ***components to be PODs***, but this is where I draw the line.

I will always prioritize ***modern C++ practices***, like correct encapsulation and abstraction. If, for instance, we take a transform component with parenting features, it becomes highly suboptimal to make it a POD with separated logic: although one might even get over the ***exposed information*** (like parent pointer, sibling pointer, ...), how do you handle ***dirty flagging***?

Let's say the user retrieves the transform which was previously marked dirty: you would need to either recalculate it immediately the moment it was requested (***pointless*** dirty flagging) ***or rely on the user*** resolving the hierarchical evaluation. I consider both options ***BAD*** design!

My engine's philosophy is that all components ***may*** apply encapsulation ***unless redundant***.

NOTES

1. One cool feature is that polled inputs linked to the ***same uid are merged*** and will be efficiently signaled at most ***once*** per frame. The signaled value is the combined result of all input events; the merging method can be ***customized using modifiers*** like ***negate*** and ***swizzle***.

2. In the example on the side, we create a mapping for the ***move action*** that creates a ***vec2*** out of the input events. Swizzle swaps to the ***Y-axis***, while negate makes the value ***negative***. This all comes down to how input values are interpreted... more on this in the next part.

INPUT ACTION MAPPING

Inspired by ***Unreal Engine***, I wanted a system to ***decouple*** input from the actual response.

The main idea is that an ***input action*** is linked to a uid which might have ***multiple bindings*** attached to it. After that, you ***subscribe to the uid*** to link an input response.

main.cpp

```
void initialize_input( input_mapping_context& imc )
{
    // move
    imc.register_input_action( input_action{ uid{ ia::move }, swizzle, negate }, KEY_w );
    imc.register_input_action( input_action{ uid{ ia::move }, swizzle }, KEY_s );
    imc.register_input_action( input_action{ uid{ ia::move }, negate }, KEY_a );
    imc.register_input_action( input_action{ uid{ ia::move } }, KEY_d );

    imc.register_input_action( input_action{ uid{ ia::move }, swizzle, negate }, DPAD_UP );
    imc.register_input_action( input_action{ uid{ ia::move }, swizzle }, DPAD_DOWN );
    imc.register_input_action( input_action{ uid{ ia::move }, negate }, DPAD_LEFT );
    imc.register_input_action( input_action{ uid{ ia::move } }, DPAD_RIGHT );

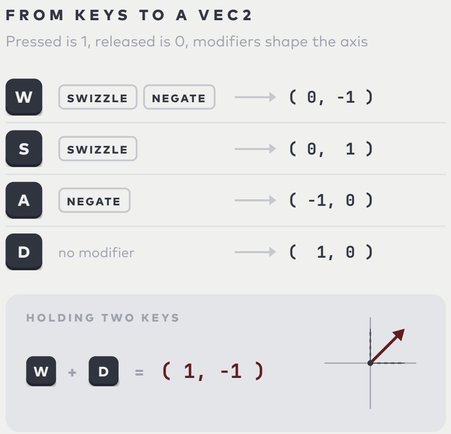
    // jump
    imc.register_input_action( uid{ ia::jump }, KEY_SPACE );
    imc.register_input_action( uid{ ia::jump }, BUTTON_A );

    // attack
    imc.register_input_action( uid{ ia::attack }, KEY_k );
    imc.register_input_action( uid{ ia::attack }, BUTTON_X );
}
```

NOTES

1. During binding, you can provide triggers. ***Pressed***, ***released*** and ***triggered*** convey at what part of the input cycle the function should be signaled.

2. I'm planning to introduce the ***command pattern*** in order to allow anonymous bindings (without the need of a controller).



FLEXIBLE INPUT VALUES

My focus was making the ***public API*** very ***clean*** and ***easy to use***. As you can see below, you just need to provide the bind function the bare minimum thanks to advanced ***template deduction***. Controller class, member function pointer ***(const/non-const)*** and method parameter type are deduced from arguments.

The bound function can have a parameter of a certain ***type*** (right now I only support ***void***, ***bool***, ***float*** and ***vec2***) and the input system will automatically convert the input value to that type.

How? An input key ***pressed*** state represents a ***1*** (***not*** directly a boolean ***true***) while ***released*** represents a ***0***. I can add them up using the previously mentioned modifiers to obtain the wanted patterns.

character_controller.cpp

```cpp
void character_controller::device_registered( input_binding_context& imc, device_info info )
{
    std::println( "CHARACTER CONTROLLER: registered device at id {}", info.id );

    imc.bind_to_input_action( this, uid{ ia::move }, &character_controller::move, pressed );
    imc.bind_to_input_action( this, uid{ ia::move }, &character_controller::move, released );
}

void character_controller::move( vec2 const dir ) const
{
    pawn_ptr_->move( dir );
}
```

A PASSION FOR GRAPHICS

Graphics programming captured my interest from the moment I built my first software raytracer. There's something magical about low-level optimization and complicated mathematics that stimulates me more than anything else.

This project embodies the peak of my graphics journey: a deep dive into ***Vulkan 1.3 API***'s complex architecture to understand hardware-accelerated graphics.

What made this extra ***interesting*** is that I made the renderer ***cross compatible with macOS***, which absolutely didn't cause me any issues at all. :)

This project is my ***Graphics Programming 2*** hand-in and it was received particularly well by the committee, achieving ***18/20***.

KEY FEATURES

- **Vulkan 1.3** dynamic rendering
- **Shader hot reload**
- **Deferred rendering** depth pre-pass
- **PBR lighting** BRDF, IBL
- **HDR post-processing** tone mapping
- **Shadow mapping**

Check it out on **GITHUB**!

VULKAN RASTERIZER

THE GRAPHICS PIPELINE

1. I started with a ***depth pre-pass***, that will populate the ***depth buffer*** while applying an early ***alpha cutout***, getting rid of transparent pixels. Below, the ***depth buffer image***.







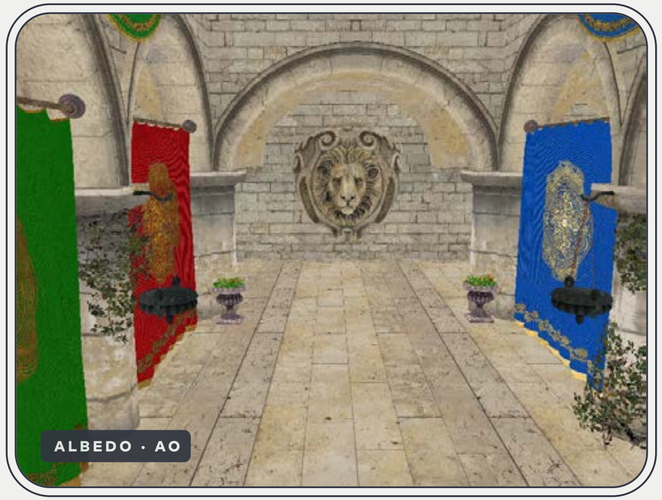
VULKAN RASTERIZER

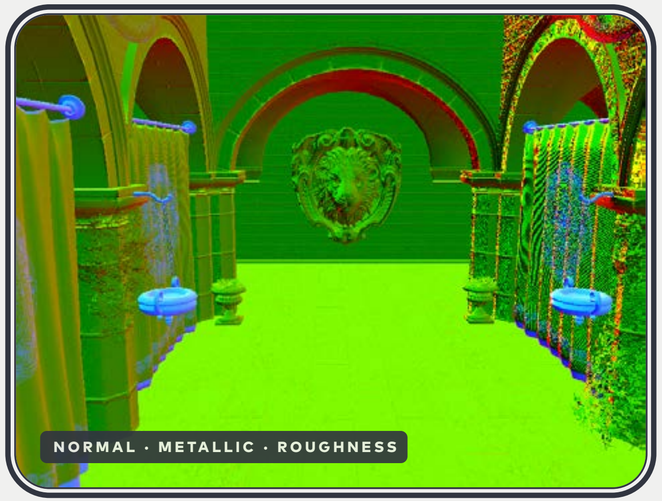
2. G-BUFFER GENERATION

... Then I proceeded with the ***G-buffer generation pass***, the main step of deferred rendering.

The G-buffer is composed of ***two images***:

- ***Albedo color (rgb)*** + ***ambient occlusion (a)***
- ***Packed normals (rg)*** + ***metallic (b)*** + ***roughness (a)***





VULKAN RASTERIZER

NOTES

1. During the command buffer recording, I wait for the depth pre-pass to finish before initiating the G-buffer generation. The differentiation between passes is merely ***conceptual*** and I find that I should probably ***combine them***, saving some synchronization time.

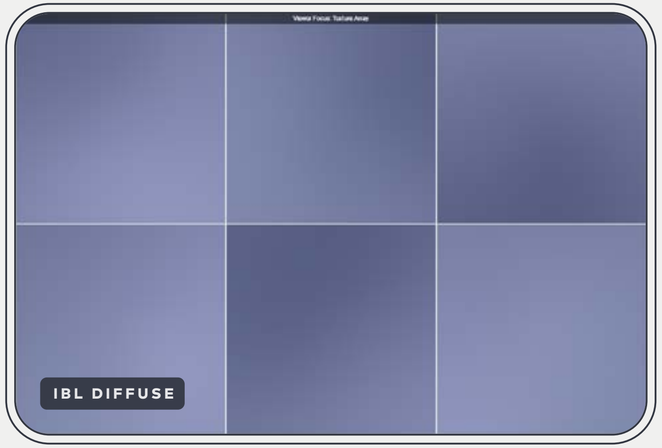
2. I use ***octahedron wrapping*** to store the normal's coordinates in 2 channels. Since normal precision is really important, I bumped up the ***channel width to 16 bits each***, leaving 32 bits used just for metalness and roughness, a bit of a ***waste***! To improve this, I'd likely transform the packed normal in a vec4 and store the rest of the information in a separate image, saving 16 bits per pixel.

3. LIGHTING PIPELINE

... Now of course the ***lighting pass*** to sample from the G-buffer images.

I process lights and sample shadow maps, to then apply BRDF and IBL. When ***outside the frustum***'s far plane, I sample from a ***cubic skybox***. My pass supports both ***directional lights*** and ***point lights***, with ***color*** and ***range*** parameters.





NOTES

1. The ***skybox cubemap***, ***shadow map*** and ***IBL diffuse cubemap*** are ***computed once*** at engine startup. If I wanted to create moving lights, I would need to setup a shadow map reconstruction with dirty flagging on the lights. This means the current POD struct setup is sub-optimal, ***unless*** I start using an ECS architecture, ***keeping the required padding*** and splitting logic elsewhere.

2. The ***IBL diffuse cubemap*** is sampled from the skybox cubemap using ***Monte Carlo Integration*** and subsequently smoothed.

3. I plan on introducing ***IBL specular cubemap*** generation as well as ***point light shadow mapping*** in the future!

4. POST PROCESSING

... And finally the ***blitting pass*** to apply a post processing chain from the previous HDR image to the final LDR present image.

Firstly, I adjust the color using a ***tone mapping function*** then adapt the exposure with a ***manual EV100 setting***.



ABOUT

A big part of this project was all the setup for the actual graphics pipeline: the ***Vulkan API setup***.

Some of the best features I implemented are:

- ***Automated device creation*** using a bitmask to select (and validate) features.
- Command pool that reuses ***vacant command buffers***.
- ***Command operator*** RAII object to aid in command buffer recording.
- ***Descriptor set allocator*** to create, access and update multiple descriptor sets with few lines of code.
- ***Many more ...***

If you're interested in ***hearing more*** about this project, don't hesitate to ***contact me***. I will happily answer any related question!

VULKAN RASTERIZER

Visit the **DAE WEBSITE** page!

A HUGE MILESTONE

This project served as my ***Programming 2*** final submission and, while the codebase is outdated, it holds particular significance in my academic journey. It received the ***highest possible grade*** and was selected as one of the exemplary course projects, ***featured on the official DAE website*** as a showcase for future students.

A lot of hard work, commitment and late-night debugging went into this project, and I'm very proud to see it showcased.



CUPHEAD REMAKE

NOTES

1. Some of the ***features*** I implemented:

- ***Animation queue*** with custom circle based collisions.
- **Camera** with ***wave based lag***, parallax, and lock-in boss fights.
- **Sprites manager** that loads each texture only once and links it to every active entity.
- **Smart enemy spawn system.**
- ***Projectiles recycling*** system.

2. I'm planning to port this entire project into my own game engine.

THE CREATIVE PROCESS

This project started from a bare-bones engine, ***only*** capable of rendering 2D images to the screen. This made it a ***constant problem-solving exercise***, and although the code is a bit rough around the edges, I'm still proud of some of the clever tricks I came up with during its development.

Even simple things like flashing the character white when damaged became a challenge since I still wasn't aware of tools like shaders at the time. For this specific problem for instance I ***didn't*** want to go the ***easy route*** and create separate color corrected sprites: I wanted to avoid such a ***redundant*** approach. What I did was copy the original texture from memory to then brighten up each individual pixel on startup.

This demonstrates one of my biggest qualities, which is the ability to approach problems in ***creative*** ways, always with a clear task in mind.

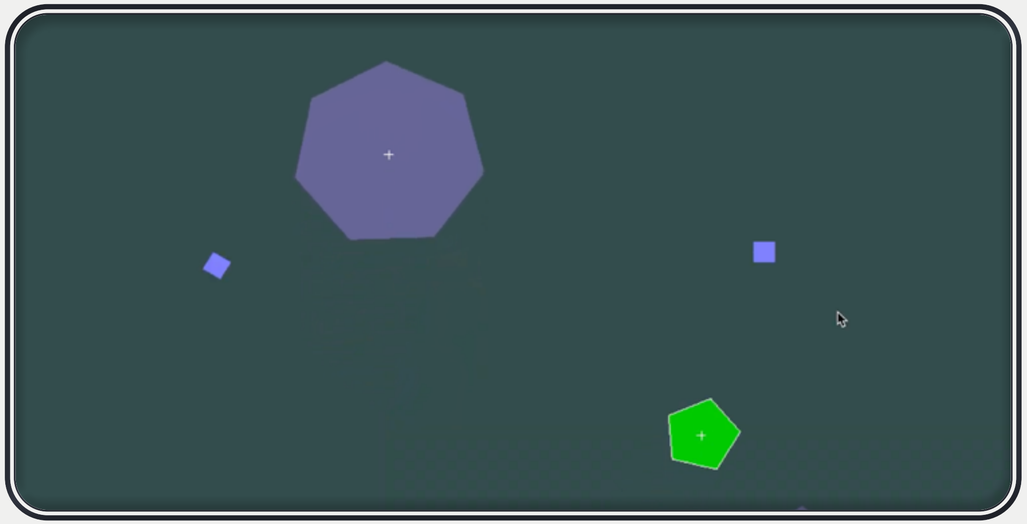


PLANE PROJECTIVE GA

This is a very niche project, using ***P***lane ***P***rojective ***G***eometric ***A***lgebra to create a physics engine. ***PPGA*** is a mathematical subject that is a further ***exploration*** of ***vectors*** and ***linear algebra***.

This project had one condition: any traditional algebra was ***forbidden***. This meant I had to figure out clever ways to ***calculate torque***, ***detect collisions*** between primitives, and ***calculate*** the ***restitution*** force... all without traditional methods.

Serving as the ***GEOA course submission***, I managed to achieve the maximum grade.



Watch the showcase **video**!

GEOA PHYSICS

CHARACTER AND VALUES

Through years of self-reflection, I've begun to understand who I am and what defines me.

Here's my best attempt at ***sincerely*** describing myself:

I'm restless in the chase for the better solution and ***can't*** settle for "good enough". I love being ***unconventional*** and ***creative***, and I want my work to stick out. If I'm given the choice between a linear path and an uncharted one, I choose the latter every single time, which is why I do best where the work needs ***high adaptability*** and ***deep thinking***.

I'm a perfectionist, trying to learn when to stop. That commitment that pushes towards quality is both my ***greatest strength*** and my ***biggest flaw***. I'm learning to understand when to ***call it a day*** and when to ***ask for help*** to avoid burning out.

I'm easy to work with. I can be shy at first, but I build friendships with colleagues quickly. ***I laugh easily and often***, but I know when to be serious. I value ***truth*** and ***manners*** above everything else.

ABOUT ME

RECOMMENDATIONS

I owe a lot to the professors and mentors who shaped my journey.

They have kindly agreed to ***serve as references*** and can speak directly of my abilities and work ethic.

Tom Tesch — the professor that fired up my passion for C++



tom.TESCH@howest.be



+32 474 975 681

Ronald van Dijk — Operations Manager at Triumph Studios



ronald.vandijk@triumphstudios.com

Stefano Prandi — my first official work employer



s.prandi@computergross.it

I additionally have his written recommendation letter at hand.